

INF3203 - Distributed Systems
Assignment 2 - Distributed log server

Jens Christian Valen Leynse

jeley4465@uit.no

Tromsø, Norway

UiT

Bjørn Helge Kværnmo

bkv005@uit.no

Tromsø, Norway

UiT

April 14, 2025

1 Introduction

The goal of the assignment is to make a server that manages log entries consistently across the entire system, while also ensuring robustness. The system should be fault-tolerant, meaning that it will continue to function even if an individual node/server experiences failures or communication issues.

The clients should be able to connect to any available server where they will put their log entries. The server is responsible for storing the values and distributing the data consistently and in order across all servers.

At its core, this is a practical implementation of the Paxos[1] consensus algorithm as described in Leslie Lamport's "Paxos Made Simple" paper. Paxos provides the foundation for keeping all servers synchronized through its efficient two-phase protocol, even when some servers temporarily fail or become unreachable. The algorithm automatically handles these real-world challenges while preserving data consistency.

2 Problem Description

The key challenge is creating a distributed log system where:

The core challenge involves building a distributed log system that maintains perfect consistency across all servers while remaining resilient to failures. Such a system must guarantee that every server stores identical log entries in the same order, ensuring clients receive consistent results regardless of which server they connect to. Crucially, the system needs to continue operating normally even when individual servers crash or become unreachable, maintaining availability for both storing and retrieving data as long as a majority of servers remain operational.

From a technical perspective, this requires ensuring safety properties where only one valid value can ever be accepted for each paxos run, while also maintaining liveness so the system continues making progress when most servers are available. The solution must properly handle failures, typically tolerating up to half of servers failing, or in other words f failures in a system with $2f+1$ servers. In practical terms, this means if you store a log entry through any server, you should later be able to retrieve that exact same entry through any other functioning server - even if some servers are temporarily unavailable. The system must automatically manage these failure scenarios without any data loss or inconsistent results, providing seamless continuity of service.

3 Method

The implementation follows Lamport's two-phase protocol as explained in the paper here [1] and this introductory youtube video [3]:

The implementation combines all three Paxos roles (proposer, acceptor, and learner) within each server node. This design choice simplifies deployment while maintaining the algorithm's correctness guarantees. Each server can initiate proposals, respond to prepare/accept messages, and learn chosen values. For log position handling, we treat each array index as a separate Paxos instance, allowing concurrent agreement on different log positions while maintaining sequential consistency within each position.

3.1 Phase 1: Prepare-Promise

- Proposer selects unique proposal number n
- Sends $\text{PREPARE}(n)$ to majority of acceptors
- Acceptor responds with:
 - $\text{PROMISE}(n, (n_a, v_a))$ where (n_a, v_a) is the highest-numbered proposal accepted
 - Or rejects if it promised to ignore proposals $\leq n$

3.2 Phase 2: Accept-Accepted

- If proposer receives **PROMISE** from majority:
 - Selects value v from highest-numbered response or new value
 - Sends **ACCEPT**(n, v) to acceptors
 - Tells learners that a consensus has been reached.
- Acceptor accepts if no promise made to higher n
- Upon acceptance by majority, value is chosen

4 Experiments and Results

4.1 Experimental Setup

We evaluated our Paxos implementation through five progressively challenging test scenarios (Table 1), designed to assess both normal operation and failure resilience in a 5-node cluster.

Table 1: Test Scenario Specifications

Scenario	Entries	Crash Frequency	Max Concurrent Crashes
0 (Baseline)	20	5-15s	1
1 (Moderate)	20	5-7s	1
2 (Intensive)	100	1-2s	1
3 (Multi-failure)	100	5-7s	2
4 (Worst-case)	100	5-7s	$N/2+1$

For the implementation we decided to switch the API handler to Flask, as we were more familiar with it, and it let us debug more efficiently. The implementation demonstrated correct safety properties where no conflicting values were ever committed, however it revealed interesting dynamics during failures. Below is a showcase of a typical crash-recovery sequence:

```
new put value: magna3586179
node1: [] [crashed]
node2: ['magna3586179'] # Healthy nodes agree
node3: ['magna3586179']
node4: ['magna3586179']
node5: ['magna3586179']
...
node1: ['magna3586179'] # Recovered and synced
```

From this we can see that:

- Crashed nodes successfully synchronized missed values upon recovery
- The system managed well with little to no crashes (Scenarios 0-2)
- Concurrent crashes (Scenarios 3-4) caused struggles with contention

4.2 Contention Challenges

The leaderless design exposed inherent contention issues during high load. The trace below shows a typical conflict scenario:

```
Proposal 174465731935002-node2 (Value: dolor9916482)
  PROMISE_RECEIVED from node1
  PROMISE_RECEIVED from node4 # Achieves quorum
Proposal 174465731525104-node4 (Value: sed130413)
  PROMISE_RECEIVED from node1 # Stalled - overwritten
```

From the node communication it is clear how node 2 first sends a proposal and receives two promises, one from node 1 and one from node 4. Then node 4 starts a new proposal with a higher ID, and the previously promised value doesn't receive any more promises, and since it failed to reach a majority it results in contention.

By following paxos logic we tried these approaches to solve the contention:

- Exponential backoff for contending proposers
- Proposal ID prioritization (higher IDs win conflicts)
- Value preservation during conflicts ("sed130413" carried forward)

Python's GIL may be a contributing factor to a performance and/or threading issues in our code. The GIL is a mechanism in Python that allows only one thread to execute at a time, which means we cannot achieve true parallelism for CPU bound tasks. This limitation will impact performance in multi-threaded environments. While the GIL typically doesn't cause significant disruptions, in our log servers, it may introduce delays when handling concurrent HTTP requests or sending messages between threads. These delays could affect log consistency and consensus latency, particularly under high load conditions [2].

5 Discussion

The system never allowed conflicting values, which means Paxos' core safety guarantee held up.

Sometimes crashed nodes still accepted writes because of what we assumed as Python's threading delays, making the logs inconsistent. Even with retries and delays, we couldn't get perfect matching between client and server logs.

We tested having a leader node first, but it just made everything more complicated without helping. Locks and queues sounded good in theory but caused more problems than they solved. The exponential backoff helped with contention but wasn't a complete fix.

The leaderless approach, while theoretically simple, presented operational challenges that align with those described in [3]. Like Torres' simplified storage system example, we found that combining the proposer/acceptor/learner roles simplified deployment but introduced contention during high load.

We simplified the promise/prepare phases by having acceptors immediately respond to prepare requests if they haven't promised a higher number, rather than maintaining a two-way promise communication network. For debugging, we implemented two key tracing tools: a debug tracer that logs internal state transitions with timestamps, and a node communication tracer that records all inter-server messages with sequence numbers. The debug tracer uses Python's logging module with a custom formatter to track proposal states, while the communication tracer intercepts all HTTP messages between nodes, storing them in an external file.

6 Conclusion

All test scenarios managed to consistently share values between servers, however they failed to gain all of the values the client logged, revealing a gap between theoretical design and our practical implementation.

While our system followed Paxos' two-phase protocol for active nodes, it lacked proper handling of transient network issues. The experimental results clearly show that while Paxos provides an excellent foundation for distributed consensus, operational deployments demand fully functional layers of fault tolerance.

Bibliography

- [1] L. Lamport, *Paxos Made Simple*, Microsoft Research, Tech. Rep. MSR-TR-2001-33, 2001. <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/paxos-simple-Copy.pdf>
- [2] D. Beazley *Understanding the Python GIL* PyCon US, Atlanta, GA, Feb. 2010. <https://www.dabeaz.com/python/UnderstandingGIL.pdf>
- [3] L. Q. Torres, *The Paxos Algorithm*, Google TechTalk, Feb. 2, 2018. <https://www.youtube.com/watch?v=JEpsBg2LTzI>